

# Claude Code 内部实践

## Skill 编写的 9 个最佳实践

原文：Tristan Hume (Anthropic) 整理：慢学 AI 频道

2026 年 4 月 23 日



---

*Lessons from Building Claude Code: How We Use Skills*

原文链接: <https://x.com/trq212/status/2033949937936085378>

---

### 摘要

本文系统梳理了 Anthropic 核心开发者 Tristan Hume 总结的 Claude Code Skill 编写九大最佳实践。这些实践围绕三个递进层次展开：**内容层**（被看见与被理解）、**结构层**（可复用与不僵化）、**高级技术层**（有状态与可组合）。通过深入剖析每个实践背后的设计哲学与具体案例，本文旨在帮助开发者从”写 Prompt 告诉 AI 怎么做”转变为”设计 Skill 让 AI 自己决定怎么做”。

# 目录

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 1   | 引言与核心问题                                               | 3  |
| 2   | 内容层——被看见与被理解                                          | 5  |
| 2.1 | 实践 1: <code>description</code> 字段是给模型看的触发条件 . . . . . | 5  |
| 2.2 | 实践 2: 不要陈述显而易见的通用知识 . . . . .                         | 6  |
| 2.3 | 实践 3: 构建 Gotchas（踩坑记录）部分 . . . . .                    | 7  |
| 3   | 结构层——可复用与不僵化                                          | 9  |
| 3.1 | 实践 4: 使用文件系统和渐进式披露 . . . . .                          | 9  |
| 3.2 | 实践 5: 仔细考虑设置流程 . . . . .                              | 10 |
| 3.3 | 实践 6: 避免过度约束 . . . . .                                | 12 |
| 4   | 高级技术层——有状态与可组合                                        | 13 |
| 4.1 | 实践 7: 内存与数据存储 . . . . .                               | 13 |
| 4.2 | 实践 8: 存储脚本并生成代码 . . . . .                             | 14 |
| 4.3 | 实践 9: 按需 Hooks . . . . .                              | 16 |
| 4.4 | 本章小结 . . . . .                                        | 18 |
| 5   | 总结与核心洞察                                               | 18 |
| 5.1 | 问题一：为什么 Skill 不被调用？ . . . . .                         | 19 |
| 5.2 | 问题二：为什么调用后效果差？ . . . . .                              | 19 |
| 5.3 | 核心洞察：Skill $\neq$ Prompt . . . . .                    | 19 |
| 5.4 | 实践检查清单 . . . . .                                      | 20 |
|     | 术语表                                                   | 22 |
|     | 视频信息                                                  | 23 |

## 1 引言与核心问题

本节首先回答一个根本问题：为什么精心编写的 Skill 常常无法达到预期效果？在 Claude Code 的生态中，开发者投入数小时构建的 Skill，往往面临三大典型困境——Agent 从不调用、调用后执行僵化、最终结果不达预期。这些痛点并非孤立的技术故障，而是反映了从”被发现”到”被正确执行”这一完整链路中的系统性挑战。



图 1: 视频封面：Claude Code 核心开发者 Tristan Hume 分享构建 Skill 的九个最佳实践

1

正如视频中 Tristan Hume 所总结的，本期内容源自 *Lessons from Building Claude Code: How We Use Skills*，其核心目标是”让 Agent 成功调用 Skill，以及如何达到更好的执行效果”。这一目标看似简单，实则涉及 Agent 决策机制、Skill 描述质量、指令设计粒度以及执行上下文管理等多个层面的协同优化。

你花两小时写了个 Skill，Agent 从来不调用；好不容易调用了，执行起来又僵化；结果不达预期。

这段描述精准概括了开发者在实际使用中的挫败感。值得注意的是，这三个问题呈现出明显的递进关系：首先是”可见性”问题——Agent 甚至不知道你的 Skill 存在；其次是”灵活性”问题——Agent 调用了 Skill，但执行过程机械死板，无法适应具体场

景；最后是”效果”问题——虽然执行了，但输出质量未能满足需求。这三层问题恰好对应了 Skill 设计的三个核心维度。



图 2: 三层问题框架示意图：内容层、结构层与高级技术层的递进关系

2

**第一层：内容层——被看见与被理解。**这一层解决的是 Agent 能否找到 Skill，以及找到后能否准确理解其用途。它对应的是 Skill 的元信息设计，包括描述（description）的清晰度、关键词的覆盖度以及与用户意图的匹配策略。如果 Agent 在决策时无法将用户的自然语言请求映射到正确的 Skill，后续一切优化都无从谈起。

**第二层：结构层——可复用与不僵化。**当 Agent 成功调用 Skill 后，下一个挑战是执行过程的灵活性。一个设计良好的 Skill 应当能够在不同场景下复用，而不是因为参数写死、路径硬编码或逻辑过于具体而失去适应能力。结构层的核心是在”确定性”与”灵活性”之间取得平衡：既要有足够的约束确保执行正确，又要保留必要的自由度以应对变化。

**第三层：高级技术层——有状态与可组合。**最高层次的问题涉及 Skill 的进阶能力：能否记住上次的执行状态（有状态性）、能否直接提供 Agent 可调用的代码片段（可组合性）、能否在必要时开启安全护栏（guardrails）。这些能力将 Skill 从简单的脚本集合提升为真正智能的、可协作的 Agent 能力单元。

这三层框架不仅是问题分类，更是 Skill 设计的指导思想。后续九条最佳实践将围绕这三个层次展开，帮助开发者系统性地提升 Skill 的可用性与执行效果。

## 2 内容层——被看见与被理解

如果说 Skill 的元数据决定了它能否进入 Claude 的”视野”，那么内容层则决定了 Claude 在调用它时能否真正”理解”意图、输出符合预期的结果。本章聚焦于 Skill 内容编写的三个核心实践，它们共同回答一个问题：如何让 Skill 的内容既被模型准确识别，又能传递团队独有的高价值信息。

### 2.1 实践 1：description 字段是给模型看的触发条件

许多开发者在编写 Skill 时，习惯性地 will description 写成功能摘要，例如”这是一个监控 PR 的工具”。这种写法看似清晰，实则误解了 description 在 Claude Code 架构中的真正作用。

正如视频中所强调的：”Claude 启动会话时会扫描所有 Skill 的 description，决定这个请求有没有对应 Skill。他需要的不是功能说明，而是触发条件。”<sup>3</sup> description 字段本质上是 Agent 进行工具检索（tool retrieval）时的匹配依据——它回答的问题是”在什么情况下应该调用这个 Skill”，而非”这个 Skill 能做什么”。



图 3: description 字段的本质是触发条件，决定 Agent 何时调用该 Skill。

4

关键区别在于语义指向：功能说明面向人类读者，描述能力边界；触发条件面向模型决策，描述调用场景。一个优秀的 description 应当包含具体的用户意图信号词。例如，

<sup>3</sup>视频时间：00:00:55

当用户说”baby-sit this PR” 或”make sure CI passes” 时，模型需要能够从 `description` 中识别出这些表达与当前 Skill 的关联。

这意味着 `description` 是 Skill 能否被调用的**第一道门槛**。如果这道门槛设错了——过于笼统导致误触发，过于狭窄导致漏触发——那么无论 Skill 内部内容多么精良，都无法在正确的时机发挥作用。建议将 `description` 视为一种”逆向 prompt”：不是你在告诉模型这个工具是什么，而是你在训练模型识别什么情况下需要它。

## 2.2 实践 2：不要陈述显而易见的通用知识

Claude 作为经过大规模预训练的大语言模型，已经掌握了海量的通用编程知识。它知道 React 的组件生命周期，了解 Python 的上下文管理器，熟悉 Git 的分支策略。在 Skill 中重复这些通用知识，不仅浪费宝贵的上下文窗口（Context Window），更会稀释真正有价值的信息密度。

视频中举了一个极具代表性的内部案例：Anthropic 的 Front-End Design Skill。工程师们发现，Claude 生成的前端界面总是默认使用 Inter 字体和紫色渐变，”看起来很 AI 味”。客户反馈不喜欢这种风格后，团队编写了一个 Skill——但这个 Skill **并不教 Claude 怎么写 React**，因为 Claude 本来就会。它只传达一件事：”别再用 Inter 和紫色渐变了，客户不喜欢。”<sup>5</sup>

---

<sup>5</sup>视频时间：00:01:25





图 4: Anthropic 内部案例：Skill 的价值在于补充团队特有的上下文与偏好，而非重复通用知识。

6

这个案例揭示了一个深刻的编写原则：**Skill 的价值在于补充 Claude 默认不知道的上下文、偏好和陷阱**。通用知识它都会，你要告诉它的是你们团队的特殊情况——你们使用的内部框架约定、客户明确排斥的设计模式、代码审查中反复出现的风格争议。这些信息才是 Skill 不可替代的核心资产。

在实际编写中，可以通过一个简单的自检问题来过滤内容：“如果我把这句话从 Skill 中删掉，Claude 是否仍然大概率能做对？”如果答案是肯定的，那么这句话很可能就是在陈述显而易见的事情，应当被删除或替换为更具特异性的指导。

## 2.3 实践 3：构建 Gotchas（踩坑记录）部分

如果说前两个实践解决了“写什么”和“不写什么”的问题，那么第三个实践则关乎“什么最值得写”。Anthropic 的内部观察发现了一个反直觉的现象：“任何 Skill 里信号最强的内容往往不是教程，而是 Gotchas。”<sup>7</sup>

<sup>7</sup>视频时间：00:02:10



图 5: Gotchas 部分是 Skill 中信号最强的内容，承载着团队真实的踩坑记录。

8

这里的”信号最强”是一个精妙的比喻。在信息论视角下，教程类内容对于 Claude 而言是低熵的——它已经在训练数据中见过大量类似的模式，因此这些信息带来的”惊讶度”很低。而 Gotchas 则是高熵信息：它们是特定团队、特定代码库、特定基础设施下的真实边界案例，是 Claude 在通用训练中几乎不可能接触到的经验。

视频中列举的 Gotchas 示例极具画面感：”不要在循环里调用 API，会慢 100 倍””记得清理临时资源，我们生产环境因为这个磁盘满过”。这些不是教科书上的最佳实践，而是用生产事故的代价换来的教训。它们的价值远超一行”注意性能优化”的泛泛之谈。

**真正值钱的是告诉他哪里别踩。**

理想情况下，Gotchas 不应当是 Skill 编写时的一次性填充，而应该是一个持续演化的知识库。随着团队在实践中不断踩到新坑、发现新的边界案例，Skill 的内容也应随之更新。这种迭代机制使得 Skill 成为团队集体经验的活化石——每一次生产故障、每一次代码审查中的意外发现，都可以转化为模型未来避免重蹈覆辙的知识约束。

从内容架构的角度，建议为每个 Skill 设立独立的 Gotchas 章节，并采用”问题现象 → 根因简述 → 正确做法”的三段式结构。这种格式既便于人类维护者快速追加新条目，也能让模型在调用时迅速定位到相关的警示信息。



内容层的三个实践构成了 Skill 编写的”黄金三角”：`description` 确保 Skill 在正确的时机被看见，避免通用知识确保内容不被噪声淹没，`Gotchas` 则提供模型无法从其他渠道获得的高价值信号。三者相辅相成，共同将 Skill 从一份普通的提示词文档，提升为团队知识工程化的核心载体。下一章将讨论结构层——当内容已经优质时，如何通过架构设计让模型更高效地消费这些信息。

## 3 结构层——可复用与不僵化

如果说第 2 章关注的是如何让 Skill 被正确理解，那么本章的核心命题则是：如何让 Skill 在真实工作流中既保持复用价值，又不沦为僵化的脚本。这涉及三个层面的设计决策：信息架构如何组织、配置状态如何管理、以及指令约束的粒度如何把握。以下三个实践分别对应这三个维度。

### 3.1 实践 4：使用文件系统和渐进式披露

许多开发者在初次编写 Skill 时，会不自觉地将所有内容塞进一个巨大的 Markdown 文件——指令、示例、模板、数据，全部堆叠在一起。这种做法看似简洁，实则严重浪费了上下文窗口（Context Window）这一稀缺资源。

正如视频中强调的：“Skill 是文件夹，不是文件。”<sup>9</sup> Claude Code 的 Skill 架构本质上是一个文件系统，这意味着你可以将不同类型的内容分离到不同的文件中，按需加载。核心逻辑保留在主文件（如 `CLAUDE.md`），详细文档放入 `references.md`，脚本模板和数据文件归入 `assets/` 目录。当 Claude 需要深入细节时，你只需告诉它”详细文档在 `references.md`，需要的时候自己去读”。

---

<sup>9</sup>视频时间：00:02:45



图 6: Skill 的文件系统组织方式：核心逻辑与详细文档分离，按需加载

10

这种设计模式在软件工程中被称为**渐进式披露**（Progressive Disclosure）——先呈现核心内容，细节在用户（或 Agent）需要时再展开加载。一个典型的应用场景是：如果 Skill 的最终输出是 Markdown 格式的报告，你可以在 `assets/` 中放置一个排版精美的模板文件，让 Claude 在生成内容后直接复制使用，而不是在指令中冗长地描述格式要求。

按需加载才能最大化利用上下文窗口。

这里的“最大化”有两层含义：一是减少单次请求中的 token 消耗，让 Claude 有更多空间处理实际任务；二是降低认知负荷——当 Claude 面对一个结构清晰的文件系统时，它能更快地定位所需信息，而不是在一个臃肿的单文件中迷失方向。

### 3.2 实践 5：仔细考虑设置流程

一个 Skill 从“可用”到“好用”的关键一跃，往往在于它是否具备状态记忆能力。很多 Skill 需要用户特定的配置信息——例如，一个站会（Stand-up）Skill 需要知道发布到哪个 Slack 频道，一个部署 Skill 需要知道目标服务器的地址。这些信息因团队而异，不可能硬编码在通用 Skill 中。

视频中的解决方案是：用 `configure.json` 存储团队特定配置，首次运行时询问，后续直接使用。<sup>11</sup>



图 7：配置流程设计：首次运行时收集团队特定信息，后续直接复用

12

这一设计让 Skill 从无状态（Stateless）工具转变为有状态（Stateful）助手。无状态的 Skill 每次运行都是一张白纸，用户需要反复提供相同的信息；而有状态的 Skill 只需配置一次，之后的每次调用都能直接复用已存储的配置。

实现这一模式时需要注意几个细节。首先，`configure.json` 应该被加入 `.gitignore`，避免敏感信息（如 Slack Webhook URL、API Key）被意外提交到版本控制。其次，首次运行的询问逻辑应当清晰明确——告诉用户“这个配置只需要设置一次，之后会自动使用”，降低用户的心理负担。最后，Skill 应当提供配置更新的机制，当团队信息发生变化时，用户可以方便地修改而不必手动编辑配置文件。

用户只需要配置一次，之后每次调用都能直接用。

这种设计哲学与 Unix 工具链中的“配置即代码”理念一脉相承：将可变的状态从不可变的逻辑中分离出来，使同一套 Skill 能够在不同环境中无缝迁移。

<sup>11</sup> 视频时间：00:03:15

### 3.3 实践 6：避免过度约束

Skill 的终极价值在于复用，而复用的前提是灵活性。一个常见的反模式是：作者在指令中规定了过于具体的执行路径，将 Skill 写成了一份死板的脚本。

视频中举了一个典型的反面教材：很多人会写”先运行 test，再运行 lint，最后运行 build”。<sup>13</sup> 这种写法的问题在于，它假设了所有场景都遵循相同的线性流程。但在实际开发中，测试、规范和构建的关系可能因项目类型、当前状态甚至用户意图而有所不同。例如，在紧急热修复场景下，开发者可能希望跳过某些非关键检查；在 CI 环境中，构建和测试可能需要并行执行以节省时间。



图 8: 约束目标而非路径：给 Claude 灵活性以适应不同场景

14

更好的写法是：”在部署前，确保代码通过测试、规范、构建，并根据情况调整顺序。”这一表述约束了目标（部署前必须通过三项检查），但将路径的选择权交给了 Claude。Agent 可以根据当前上下文判断最优执行顺序——如果测试耗时较长而 lint 能快速发现问题，它可能会选择先运行 lint；如果项目配置了并行构建，它可以同时触发多个任务。

约束目标，而不是约束路径。给 Claude 他需要的信息，但也要给他适应不同情况的灵活性。

这一原则与自动驾驶领域的”操作约束”（Operational Constraints）概念相似：系统被告知”不要撞到行人”（目标约束），而不是”在每秒 30 帧时向左转 15 度”（路径约

<sup>13</sup>视频时间：00:03:35



束)。前者在复杂环境中具有更强的鲁棒性，因为它允许系统根据实时感知选择最优动作。

将 Skill 写成死板的脚本，本质上是将作者当前对问题的有限理解强加于未来的所有场景。而优秀的 Skill 设计应当承认不确定性，通过约束目标、释放路径，让 Claude 在面对未知情境时仍能做出合理的决策。这不仅提升了 Skill 的复用范围，也使其在面对边缘情况时表现得更加智能和稳健。

---

本章的三个实践共同指向一个核心主题：**Skill 的设计应当服务于 Agent 的智能，而非限制它**。通过文件系统的渐进式披露，我们为 Agent 提供了高效获取信息的基础设施；通过配置流程的状态化设计，我们让 Skill 能够适应不同的使用环境；通过避免过度约束，我们保留了 Agent 在复杂情境中自主决策的空间。这三者相辅相成，构成了 Skill 结构层的完整设计范式。

## 4 高级技术层——有状态与可组合

如果说前六条实践构成了 Skill 编写的地基，那么最后三条则属于高级技术层，它们解决的核心问题是：**Skill 运行时的能力建设**。当 Skill 从简单的单次调用脚本，演进为需要跨会话协作、持续追踪状态、动态组合能力的智能体组件时，设计者必须面对三个根本挑战——如何赋予无状态运行以记忆？如何避免重复造轮子？如何在特定场景下安全地启用严格规则而不造成全局负担？本章将围绕实践七至九，深入探讨有状态化、可组合性与按需治理的设计哲学。

### 4.1 实践 7：内存与数据存储

Claude Code 的 Skill 机制有一个基本设定：**每次运行都是全新的**。这意味着 Skill 在默认情况下是无状态的（Stateless）——它不记得上次做了什么，不保留任何中间结果，也不会继承前一次对话中的上下文。这种设计保证了运行的隔离性与可预测性，但在许多真实场景中却构成了显著的障碍。例如，一个用于监控代码变更的 Skill，如果每次都要重新扫描整个仓库，不仅效率低下，也无法向用户呈现“自上次检查以来的新增问题”。

Anthropic 提出的解决方案简洁而优雅：**让 Skill 写日志**。具体而言，Skill 在每次运行结束后，将关键信息以追加（append）模式写入持久化存储——可以是一个本地日志文件、一个 JSONL 记录，或一个专门的摘要文件。下一次运行时，Skill 首先读取这份日志，了解“上次做到了哪里”，然后在此基础上只报告新的变化。

这一机制的本质，是将外部存储转化为 Skill 的“外置记忆体”。正如视频中提到的：“*skill* 不再是无状态的脚本，而是有记忆的助手。”这种有状态化（Stateful）的转变，使得 Skill 能够胜任需要持续追踪的任务场景：



- **增量审计**：代码安全扫描 Skill 记录上次检查的 commit hash，下次只审计新增提交；
- **进度追踪**：长周期数据处理任务记录已完成的批次编号，支持断点续传；
- **状态摘要**：项目管理 Skill 维护一个持续更新的 TODO 摘要文件，每次运行先读摘要、再追加新发现。

值得注意的是，日志的设计应当遵循**只追加、不修改**的原则。这不仅保证了审计追踪的完整性，也避免了并发运行时的写冲突。一个好的实践是将日志格式设计为结构化数据（如 JSON Lines），每条记录包含时间戳、运行摘要和关键状态字段，便于后续解析与过滤。



图 9: 实践 7：通过追加日志实现跨运行记忆，让无状态 Skill 获得有状态能力

15

**核心洞见**：无状态是基础设施的默认假设，有状态是业务需求的刻意设计。Skill 的记忆不依赖运行时保留，而依赖退出时的持久化与启动时的恢复。

## 4.2 实践 8：存储脚本并生成代码

在 Claude Code 的日常使用中，一个常见的反模式是：Agent 面对重复性任务时，每次都从零开始编写相似的样板代码。这不仅浪费时间，还容易引入不一致性——同样的功能在不同会话中可能以略微不同的方式实现，导致维护困难。

实践八的核心主张是：**别让 Claude 每次都从头写样板代码**。具体策略是将稳定、可复用的能力封装为脚本（scripts）和辅助函数（helper functions），存储在 Skill 的目录结构中，让 Agent 负责调用与组合，而不是重复创造。

这一理念与软件工程中的**可组合性**（Composability）原则一脉相承。可组合性要求将复杂系统拆分为独立、可复用的单元，上层应用根据具体场景动态组合这些单元。在 Skill 的语境下，这意味着：

- **脚本化稳定流程**：将环境检测、依赖安装、测试运行等标准化流程写成可执行脚本，Agent 只需调用 `./scripts/setup.sh` 而非逐行生成命令；
- **函数化通用逻辑**：将文件过滤、正则匹配、API 请求封装为 Python 或 Bash 函数，放入 `lib/` 或 `helpers/` 目录，供 Agent 在生成代码时直接引用；
- **模板化常见输出**：为报告、配置文件、代码片段预定义模板，Agent 填充变量即可，避免每次重新排版。

视频中有一句精辟的总结：“*Claude Code* 最好的工具不是更多文档，而是可调用的代码。”这句话揭示了 Agent 时代的工具观转变——对于能够阅读、理解并执行代码的 AI 来说，一段可直接调用的脚本远比一份需要人工翻译的文档更有价值。文档描述“应该怎么做”，而代码直接表达“可以怎么做”。当 Agent 面对一个已封装好的函数时，它不需要理解实现细节，只需知道输入输出契约即可正确调用。



图 10: 实践 8：将稳定能力封装为可调用脚本与辅助函数，Agent 负责组合而非重造轮子

16

从架构角度看，这种设计将 Skill 的能力分层：底层是稳定的、经过测试的脚本与函数库，上层是 Agent 根据用户意图动态生成的编排逻辑。底层越厚实，上层的生成任务就越轻量，出错概率也越低。这与传统软件开发中“库 + 应用”的分层模式异曲同工，只是这里的“应用层”由大语言模型在运行时动态生成。

**设计原则：**凡是出现三次以上的相似代码，都值得考虑封装为 Skill 内部的共享脚本或函数库。Agent 的创造力应当用于解决新问题，而非重复解决旧问题。

### 4.3 实践 9：按需 Hooks

在软件系统的治理中，安全规则与操作约束往往面临一个两难：规则太松则风险不可控，规则太严则日常操作处处受限。全局开启最严格的检查策略，固然能防止灾难性错误，但也会严重拖累开发效率，导致用户为了绕过规则而采取更危险的变通手段。

实践九提出的**按需 Hooks**机制，正是为了解决这一矛盾。Hook 是在特定生命周期事件（如命令执行前、文件写入前、API 调用前）插入的回调机制。与全局静态规则不同，按需 Hook 的核心特征是**动态注册、会话隔离、自动失效**。

视频中以 `careful` Skill 为例进行了说明。当用户调用 `careful` 时，该 Skill 会注册一个命令执行前的 Hook，用于拦截危险操作——例如 `rm -rf /`、`DROP TABLE` 等可能

导致不可逆损失的命令。这个 Hook 只在当前会话中有效，会话结束后自动失效，不会对其他会话或全局环境造成任何影响。

这种设计带来了三个显著优势：

1. **精准治理**：只在需要的场景启用严格规则。执行数据库迁移时开启 `careful`，日常开发时保持默认宽松策略；
2. **零残留风险**：会话级 Hook 随会话结束而销毁，不会因为忘记关闭规则而影响后续操作；
3. **可组合的安全策略**：不同 Skill 可以注册不同类型的 Hook，形成针对特定任务的安全防护网，而非一刀切的全局规则。

从技术实现上看，按需 Hook 通常依赖于宿主环境（如 Claude Code CLI）提供的事件总线或中间件机制。Skill 通过声明式配置或运行时 API 注册回调函数，宿主在触发相应事件时按优先级调用这些回调。回调可以决定是否允许操作继续执行、是否需要用户确认、或者是否应当修改操作参数。



图 11: 实践 9：通过会话级 Hook 在特定场景下启用严格规则，会话结束后自动失效

17

按需 Hook 的设计哲学可以概括为：**权力下放，责任上移**。系统将注册 Hook 的能力开放给各个 Skill，但由宿主环境保证 Hook 的生命周期管理和执行隔离。这既赋予了 Skill 强大的场景化治理能力，又避免了规则泛滥导致的系统复杂化。



**安全启示：**最好的安全策略不是阻止一切风险操作，而是在正确的时机为正确的人提供正确的保护。按需 Hook 让安全成为一种可组合的服务，而非不可逾越的围墙。

## 4.4 本章小结

实践七至九共同指向一个更高阶的设计目标：让 Skill 从简单的命令封装，进化为具备记忆、可复用、可治理的智能体组件。通过外部日志实现有状态化，通过脚本库实现可组合性，通过按需 Hook 实现精细化治理——这三者构成了 Skill 高级技术层的铁三角。掌握它们，意味着 Skill 设计者已经超越了“让 AI 能做事”的初级阶段，进入了“让 AI 持续、高效、安全地做事”的成熟境界。

## 5 总结与核心洞察

经过前四章对九条最佳实践的系统梳理，我们回到本章开篇提出的根本问题：为什么精心编写的 Skill 常常不被调用？为什么调用之后效果仍然不尽如人意？视频结尾的总结段落以极其凝练的方式揭示了这两个问题的共同根源——不要把 Skill 当成 Prompt 来写。

**TAKEAWAY** 慢学AI bilibili

### 好的Skill不是prompt，而是组合能力

回到开头的问题：为什么你的Skill不被调用？为什么调用后效果差？

- 不被调用**  
大概率是因为你把Description写成了摘要，而不是触发时机
- 效果差**  
大概率是因为你写了一堆Claude已经知道的东西，或者步骤写得太死

**核心观点**  
好的Skill不是告诉Claude每一步做什么，而是给它组合的能力

**从prompt到工具包**  
给Claude工具，而不是给它指令。让它在执行时自己决定怎么做。

为什么调用后效果差？

6/6

图 12: 核心洞察：Skill 的本质是赋予 Agent 组合能力，而非规定固定执行路径



## 5.1 问题一：为什么 Skill 不被调用？

正如视频中所一针见血指出的：“不被调用，大概率是因为你把 *description* 写成了摘要，而不是触发时机。”<sup>19</sup> 这一判断与第 2 章实践 1 的分析完全呼应——*description* 字段在 Claude Code 的架构中承担着工具检索（tool retrieval）的关键职能，它回答的问题是“在什么情况下应该调用这个 Skill”，而非“这个 Skill 能做什么”。

当开发者习惯性地 *description* 写成功能摘要时，Agent 在扫描决策阶段无法建立“用户意图 → Skill 能力”的有效映射。即使用户的请求与 Skill 的实际能力高度匹配，Agent 也因为缺乏明确的触发信号而选择忽略它。这解释了为什么许多开发者会困惑于“我的 Skill 明明很有用，但 Claude 从来不调用”——问题不在于 Skill 的内容质量，而在于元数据设计的语义错位。

回顾第 2 章的论述，解决这一问题的核心策略是将 *description* 视为一种“逆向 prompt”：不是你在告诉模型这个工具是什么，而是你在训练模型识别什么情况下需要它。只有当 *description* 中包含了具体的用户意图信号词——如“baby-sit this PR”“make sure CI passes”“deploy to production”——Agent 才能在正确的时机做出正确的路由决策。

## 5.2 问题二：为什么调用后效果差？

视频对此同样给出了精准诊断：“效果差，大概率是因为你写了一堆 Claude 已经知道的东西，或者步骤写得太死。”<sup>20</sup> 这两个原因分别对应了第 2 章实践 2 和第 3 章实践 6 所揭示的反模式。

**重复通用知识**的问题在于稀释了信息密度。Claude 作为经过大规模预训练的模型，已经掌握了 React 组件生命周期、Python 上下文管理器、Git 分支策略等通用编程知识。当 Skill 中充斥着这些显而易见的内容时，真正有价值的团队特有信息——如内部框架约定、客户排斥的设计模式、生产事故的教训——就会被淹没在噪声之中。第 2 章中 Anthropic 的 Front-End Design Skill 案例正是最佳反面教材：它不教 Claude 怎么写 React，只传达“别用 Inter 字体和紫色渐变，客户不喜欢”。

**步骤写得太死**的问题则在于剥夺了 Agent 的自主决策空间。当 Skill 将执行路径规定得过于具体——如“先运行 test，再运行 lint，最后运行 build”——Agent 就失去了根据具体场景灵活调整的能力。第 3 章实践 6 强调的核心原则是“约束目标，而不是约束路径”：告诉 Agent“部署前必须通过测试、规范和构建”，但将执行顺序的选择权交给它，让它根据当前上下文判断最优策略。

## 5.3 核心洞察：Skill ≠ Prompt

视频最后抛出了本期内容的核心观点：

---

<sup>19</sup> 视频时间：00:05:15

<sup>20</sup> 视频时间：00:05:25

好的 Skill 不是告诉 Claude 每一步做什么，而是给他组合的能力，让他在执行时自己决定怎么做。

这句话揭示了 Skill 与 Prompt 在本质上的分野。Prompt 是一种直接指令——作者预设了完整的执行路径，模型只需按部就班地跟随。而 Skill 是一种能力赋权——作者提供的是可复用的组件、约束条件和上下文信息，模型则根据具体场景自主决定如何组合与调度这些能力。

这一洞察与前四章的九条实践形成了完整的认知闭环：

- **内容层**（实践 1–3）解决“被看见”和“传递高价值信号”的问题，确保 Skill 在正确时机被调用，且传递的是 Claude 无法从通用训练中获得的信息；
- **结构层**（实践 4–6）解决“高效消费”和“灵活执行”的问题，通过文件系统的渐进式披露、配置状态化和目标约束释放 Agent 的认知资源与决策空间；
- **高级技术层**（实践 7–9）解决“持续演进”和“可组合治理”的问题，让 Skill 具备跨运行记忆、代码级复用能力和场景化安全策略。

三者层层递进，共同服务于同一个终极目标：让 Agent 更智能，而不是让脚本更复杂。

## 5.4 实践检查清单

为了将九条最佳实践转化为可落地的行动指南，以下提供一份编写 Skill 时的自检清单。在发布或共享你的 Skill 之前，建议逐条核对：

1. **description 是触发条件吗？** 删除功能摘要式的描述，替换为“当用户说 X 或需要做 Y 时，调用此 Skill”的句式。
2. **内容中有通用知识吗？** 逐句自检：删掉这句话后，Claude 是否仍然大概率能做对？如果是，则删除或替换为团队特有信息。
3. **有 Gotchas 章节吗？** 记录至少三个团队真实踩过的坑，采用“问题现象 → 根因简述 → 正确做法”的三段式结构。
4. **使用了文件系统组织吗？** 核心逻辑在主文件，详细文档放入 `references.md`，模板和数据归入 `assets/` 目录。
5. **考虑过设置流程吗？** 团队特定配置（频道、地址、偏好）通过 `configure.json` 存储，首次询问、后续复用。
6. **约束的是目标还是路径？** 将“先 A 再 B 再 C”改写为“确保完成 A、B、C，顺序由你根据场景决定”。

7. **需要跨运行记忆吗？**若 Skill 需要追踪状态，设计追加日志机制，下次运行先读日志、再报告新变化。
8. **有可调用的代码吗？**将重复出现的流程封装为脚本或辅助函数，让 Agent 调用而非从头生成。
9. **安全规则是按需的吗？**避免全局静态规则，优先使用会话级 Hook，在特定场景下动态启用、自动失效。

---

九条最佳实践，三条核心洞察，一个根本转变：从”写 Prompt 告诉 AI 怎么做”到”设计 Skill 让 AI 自己决定怎么做”。这不仅是一种技术范式的演进，更是对智能体（Agent）本质的重新理解——最好的工具不是最详细的说明书，而是最能激发使用者创造力的能力组合。当 Skill 设计者放下对控制权的执念，转而为 Agent 铺设清晰的能力边界与丰富的组合素材时，真正的智能协作才刚刚开始。

## 术语表

### Agent

具备自主决策能力的 AI 系统，能根据上下文选择工具并执行多步任务。

### Claude Code

Anthropic 推出的 AI 编程助手 CLI，支持通过 Skill 扩展 Agent 能力。

### Composability（可组合性）

将能力拆分为独立、可复用的单元，Agent 根据场景动态组合。

### Context Window（上下文窗口）

模型一次能处理的最大 token 数量。

### Description

Skill 元数据中的描述字段，用于 Agent 扫描决策，决定是否在特定用户请求下触发该 Skill。

### Gotchas

团队在实践中真实踩过的坑、边界案例与注意事项，通常比通用教程更具信号强度。

### Hook

在特定生命周期事件（如命令执行前）插入的回调机制，用于动态启用规则或拦截危险操作。

### Prompt

直接指示模型执行某任务的指令文本；与 Skill 的区别在于 Skill 强调可复用的能力组合。

### Progressive Disclosure（渐进式披露）

信息架构设计原则，先呈现核心内容，细节在用户需要时再展开加载。

### Skill

Claude Code 中的可复用能力单元，由描述、指令、脚本和配置组成，供 Agent 在适当时机调用。

### Stateful（有状态）

能够记忆和复用之前的配置、日志或执行结果，实现跨会话的连续性。

### Stateless（无状态）

每次运行独立，不保留上次执行的信息或配置。

### Trigger Condition（触发时机）

描述 Skill 应在何种用户请求或场景下被调用的条件语句。



## 视频信息

- 标题：Claude Code 内部实践：Skill 编写的 9 个最佳实践
- 原文标题：Lessons from Building Claude Code: How We Use Skills
- 演讲者：Tristan Hume (Anthropic)
- 频道：慢学 AI
- Bilibili: <https://www.bilibili.com/video/BV1kxQBB6E86>
- 时长：5 分 41 秒
- 整理日期：2026 年 4 月 23 日